AULA PRÁTICA • .NET + JAVASCRIPT

DO FRONTEND AO BACKEND

Como conectar o projeto Mini DataSales a uma API em C#

VS Code / HTML + CSS + JavaScript / ASP.NET Core

Não vamos começar outro projeto

Aproveitaremos o frontend que ja foi desenvolvido.

O que já temos

Uma interface com indicadores, tabelas, estilo visual e lógica em JavaScript.

O que faremos agora

Uma API em .NET que fornece dados reais para a interface — inicialmente simulados em memória.

Meta: trocar valores fixos por informações recebidas do backend.

O que vocês conseguirão fazer

Ao final, o frontend vai conversar com o C#.

- Criar rotas GET no .NET
- Retornar informações em JSON
- Consumir a API com fetch() no JavaScript
- Atualizar os indicadores do HTML
- Testar mudanças nos dados sem reescrever a interface

Nossa aula em etapas

Um percurso gradual: construir, testar e integrar.

01 **Preparar o projeto**

20–30 min

02 **Criar a API**

40 min

03 **Trabalhar com dados**

40 min

04 **Conectar via fetch**

50 min

05 **Aplicar no projeto**

50 min

06 **Exercício e testes**

40 min

Como as três partes se conectam

Uma requisição sai do navegador e uma resposta volta.



Navegador → GET /api/indicadores → cálculo em C# → JSON → tela atualizada

Organizando a estrutura de pastas

HTML e API funcionarão no mesmo endereço.

```
MiniDataSales/
│
├── Program.cs              ← backend (C#)
├── MiniDataSales.csproj    ← configuração .NET
└── wwwroot/                ← frontend público
    ├── index.html
    ├── css/
    │   └── style.css
    └── js/
        └── script.js
```

Por que wwwroot?

O .NET expõe arquivos estáticos dessa pasta.

Atenção

Não é necessário refazer o layout. Basta mover os arquivos.

Criando o projeto pelo terminal

Se o projeto .NET já existe, use a pasta dele e pule a criação.

```
dotnet new web -n MiniDataSales
cd MiniDataSales
code .
```

O comando dotnet new web

Cria uma aplicação web mínima em ASP.NET Core.

Depois

Crie wwwroot e copie o frontend para dentro dela.

Configurando a aplicação

No Program.cs, começamos com o servidor e os arquivos estáticos.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

UseDefaultFiles procura index.html • UseStaticFiles permite servir CSS, JS e HTML.

Vamos simular cinco vendas

Por enquanto, nada de banco de dados: os registros ficam na memória.

```
var vendas = new[]
{
    new { Produto = "Notebook", Categoria = "Tecnologia", Valor = 3500m },
    new { Produto = "Mouse", Categoria = "Tecnologia", Valor = 100m },
    new { Produto = "Cadeira", Categoria = "Moveis", Valor = 600m },
    new { Produto = "Mesa", Categoria = "Moveis", Valor = 900m },
    new { Produto = "Monitor", Categoria = "Tecnologia", Valor = 1200m }
};
```

O sufixo m indica decimal em C#: adequado para os valores deste exemplo.

Criando nossa primeira rota GET

Inserir antes de app.Run().

```
app.MapGet("/api/vendas", () => vendas);
```

MapGet

Define uma rota que atende solicitações HTTP do tipo GET.

/api/vendas

É o endereço que retornará a lista de registros em JSON.

Calculando informações com C#

Agora o backend calculará os valores que a tela precisa.

```
app.MapGet("/api/indicadores", () =>
{
    var receitaTotal = vendas.Sum(v => v.Valor);
    var totalVendas = vendas.Length;
    var ticketMedio = receitaTotal / totalVendas;

    return new { receitaTotal, totalVendas, ticketMedio };
});
```

Receita total = soma dos valores • Ticket médio = receita / número de vendas

Interpretando o código

Não decore: entenda a responsabilidade de cada linha.

`new[]`	Cria uma coleção de registros.
`MapGet()`	Expõe uma rota que pode ser acessada pelo navegador.
`Sum()`	Soma os valores selecionados.
`Length`	Informa quantos registros há na coleção.
`return new { ... }`	Define os dados que serão devolvidos em JSON.

Como verificar se a API funciona

Primeiro testamos o backend sozinho, depois conectamos a página.

```
dotnet run
```

No terminal, procure uma linha semelhante a:

```
Now listening on: http://localhost:5000
```

> **Teste no navegador**
>
> Abra http://localhost:5000/api/vendas (ajuste a porta conforme o terminal).

O que o navegador recebe?

A API não precisa devolver HTML: ela envia dados estruturados.

```
[
  { "produto": "Notebook", "categoria": "Tecnologia", "valor": 3500 },
  { "produto": "Mouse", "categoria": "Tecnologia", "valor": 100 }
]
```

JSON = formato de troca de dados entre sistemas. Este trecho mostra só duas vendas.

Preparando os elementos do HTML

Mantenha o CSS. Acrescente IDs aos valores que serão atualizados.

```
<h3>Receita Total</h3>
<p id="receita">R$ 0,00</p>

<h3>Total de Vendas</h3>
<p id="total">0</p>

<h3>Ticket Médio</h3>
<p id="ticket">R$ 0,00</p>
```

O id é o identificador que permite ao JavaScript localizar o elemento.

Agora vem a conexão: fetch()

Adicionar a função no script.js já existente.

```
async function carregarIndicadores() {
    const resposta = await fetch("/api/indicadores");
    const dados = await resposta.json();

    document.getElementById("receita").textContent =
        "R$ " + dados.receitaTotal.toFixed(2);

    document.getElementById("total").textContent = dados.totalVendas;
    document.getElementById("ticket").textContent =
        "R$ " + dados.ticketMedio.toFixed(2);
}
carregarIndicadores();
```

O que acontece dentro de fetch()?

Quatro etapas que se repetem em aplicações web.

01	**SOLICITAÇÃO**	JavaScript pede dados para /api/indicadores
02	**PROCESSAMENTO**	O C# executa os cálculos das vendas
03	**RESPOSTA**	A API devolve os números em JSON
04	**ATUALIZAÇÃO**	O JavaScript modifica o conteúdo do HTML

RESULTADO

Nosso dashboard recebe os indicadores

Valores calculados pelas cinco vendas do exemplo.

RECEITA TOTAL	TOTAL DE VENDAS	TICKET MÉDIO
R$ 6.300,00	5	R$ 1.260,00

Antes: números definidos no frontend. Agora: números enviados pelo backend.

Mudou o dado? A interface deve mudar

Vamos provar que o backend é a fonte das informações.

1. No Program.cs, altere o Notebook de 3500m para 5000m.
2. Pare o servidor (Ctrl + C) e execute dotnet run novamente.
3. Atualize a página e observe a nova receita.

Pergunta: por que a interface mudou mesmo sem alterar o HTML?

Checklist rápido de diagnóstico

A maior parte dos erros iniciais envolve caminho, rota ou identificador.

404 na rota	Conferir MapGet e o endereço digitado.
HTML abre sem CSS	Verificar os caminhos dentro de wwwroot.
Indicadores zerados	Conferir IDs e o carregamento do script.
fetch falha	Abrir F12 → Console e verificar o erro.
Porta diferente	Copiar exatamente a porta mostrada no terminal.
Valor antigo	Reiniciar o backend e atualizar a página.

Desafio: evoluir o seu Mini DataSales

Agora cada aluno aplica o aprendizado ao frontend que já possui.

- Manter o próprio layout e integrar ao projeto .NET.
- Criar uma rota com pelo menos cinco registros.
- Conectar pelo menos um indicador com fetch().
- Adicionar duas vendas novas no backend.
- Comprovar a atualização dos valores na interface.
- Personalizar um indicador ou informação exibida.

Conseguimos integrar as duas camadas

Hoje a tela passou a exibir informações produzidas no servidor.

O que aprendemos

GET, rotas em C#, JSON, fetch(), atualização de elementos HTML e teste local.

Próxima aula

Criar uma rota POST e um formulário para cadastrar novas vendas.

Pergunta final: o que mudaria se as vendas viessem de um banco de dados?

DO CÓDIGO À APLICAÇÃO REAL.

O frontend mostra. O backend processa.
A integração faz tudo funcionar.

PRÓXIMO PASSO: CADASTRAR VENDAS COM POST